

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	P Darshith	Reg. No.:	192571051
Section / Batch:	2025	Date:	28-07-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1

SECTION A – APPLICATION BASED QUESTIONS

Q77. Selection Sort – Dataset with Two Smallest Equal Values

Apply the Brute Force technique using Selection Sort on the dataset [24, 7, 31, 7, 45, 12]. Clearly interpret the problem and justify why Selection Sort is suitable for identifying the minimum element in each pass. Perform the sorting step-by-step and show the intermediate arrays after each pass. Count the number of comparisons and swaps made during execution. Analyze the time complexity in all cases. Verify the correctness of the final sorted output and interpret the behavior of the algorithm on equal minimum values.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ANSWER:-

Dataset with Two Smallest Equal Values

Given the dataset [24, 7, 31, 7, 45, 12], the Selection Sort algorithm is used to arrange the elements in ascending order. Selection Sort is a simple brute-force sorting technique that repeatedly selects the smallest element from the unsorted portion of the array and places it in its correct position. The algorithm continues this process until all elements are sorted. It is suitable for this problem because it directly searches for the minimum element during each pass, making the sorting process easy to understand and implement.

The given array contains two equal minimum values, **7 and 7**. This allows us to observe how Selection Sort handles duplicate values while sorting.

The initial array is:

[24, 7, 31, 7, 45, 12]

During the first pass, the algorithm searches the entire array and finds the smallest element, which is 7. This value is swapped with the first element, 24.

After Pass 1:

[7, 24, 31, 7, 45, 12]

In the second pass, the algorithm considers the remaining unsorted elements [24, 31, 7, 45, 12]. The smallest element is again 7, which is swapped with 24.

After Pass 2:

[7, 7, 31, 24, 45, 12]

In the third pass, the unsorted portion becomes [31, 24, 45, 12]. The minimum element is 12, which is swapped with 31.

After Pass 3:

[7, 7, 12, 24, 45, 31]

In the fourth pass, the unsorted portion is [24, 45, 31]. Since 24 is already the smallest element, no swap is required.

After Pass 4:

[7, 7, 12, 24, 45, 31]

In the fifth pass, the remaining unsorted elements are [45, 31]. The smallest element is 31, which is swapped with 45.

After Pass 5 (Final Sorted Array):

[7, 7, 12, 24, 31, 45]

The intermediate arrays obtained after each pass are shown below:

Pass Array

Initial [24, 7, 31, 7, 45, 12]

Pass 1 [7, 24, 31, 7, 45, 12]

Pass 2 [7, 7, 31, 24, 45, 12]

Pass 3 [7, 7, 12, 24, 45, 31]

Pass 4 [7, 7, 12, 24, 45, 31]

Pass 5 [7, 7, 12, 24, 31, 45]

The number of comparisons made by Selection Sort can be calculated as:

$$(n-1) + (n-2) + (n-3) + \dots + 1$$

For **n = 6**:

$$5 + 4 + 3 + 2 + 1 = 15 \text{ comparisons}$$

Thus, the algorithm performs **15 comparisons**.

The swaps performed are:

- Swap 24 and 7
- Swap 24 and 7
- Swap 31 and 12
- No swap required
- Swap 45 and 31

Therefore, the total number of swaps is **4**.

Selection Sort always searches the unsorted portion of the array to find the minimum element, regardless of the initial arrangement of data. Hence, its time complexity remains the same in all cases.

Best Case: $O(n^2)$

Average Case: $O(n^2)$

Worst Case: $O(n^2)$

The space complexity of Selection Sort is **$O(1)$** because it sorts the array in-place without requiring additional memory.

The final sorted output obtained is:

[7, 7, 12, 24, 31, 45]

This output is correct because every element is arranged in ascending order, and all values from the original array are present in the final result. The duplicate minimum values (**7 and 7**) are placed next to each other at the beginning of the sorted array.

When equal minimum values occur, Selection Sort selects the first occurrence of the minimum element found during scanning. Although the algorithm correctly sorts duplicate values, it is generally considered **not stable**, because the relative order of equal elements may change due to swapping. However, in this example, both values are identical and appear correctly in the final sorted sequence.

In conclusion, Selection Sort successfully sorts the dataset **[24, 7, 31, 7, 45, 12]** by repeatedly selecting the minimum element and placing it in the correct position. The algorithm performs **15 comparisons** and **4 swaps**, producing the final sorted array **[7, 7, 12, 24, 31, 45]**. While Selection Sort is simple and easy to understand, its **$O(n^2)$** time complexity makes it inefficient for large datasets. Nevertheless, it remains a useful algorithm for learning the fundamentals of sorting and the brute-force approach.